

UNIT-II

Data Structures and Algorithms

Short Answer Type Questions

1. What is a list in Python?

LIST:

In Python, a list is a built-in dynamic sized array (automatically grows and shrinks). It allows you to store an ordered collection of items.

- List can store different data types.
- List can contain duplicate items.
- List in Python are Mutable. Hence, we can modify, replace or delete the items.
- List are ordered. It maintain the order of elements based on how they are added.
- Accessing items in List can be done directly using their position (index), starting from 0.

```
a=[10,20,30] #LIST
print(a)
a. append(21)
a.extend([50, 60])
a.insert(2, 25)
print("After Additions:", a)
```

2. How is a tuple different from a list in Python?

TUPLE:

A tuple in [Python](#) is an immutable ordered collection of elements. Tuples are similar to lists, but unlike lists, they cannot be changed after their creation (i.e., they are immutable). Tuples can hold elements of different data types. The main characteristics of tuples are being ordered , heterogeneous and immutable.

```
#create using () round brackets
a=(1,2,3,4,5)
print(a)
print(type(a))
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
(1, 2, 3, 4, 5)
<class 'tuple'>
```

3. Define a dictionary with an example.

A **Python dictionary** is a data structure that stores the value in **key: value** pairs. Values in a dictionary can be of any data type and can be duplicated, whereas keys can't be repeated and must be *immutable*.

```
# create dictionary using { }
d= {"name": 'john', "Rollno": 1234, "marks": 98.5}
print(d)

# Access using key
print(d["name"])

# Access using get(): pass key as an argument
print(d.get("rollno"))

# Adding a new key-value pair
d["section"] = "DIVISION-6"

# Updating an existing value
d["marks"] = 95.2 # pass the name of the key to update its value
print(d)
```

4. What is the purpose of the set() function in Python?

A **Set** in Python is used to store **unordered** collection of **unique** items

Empty Set Can be created using set() constructor method:

```
a=set()# since a={} is considered as an empty dictionary not an empty set.  
print(type(a))  
PS C:\Users\masrath parveen\onedrive\desktop> python f.py  
<class 'set'>
```

The Python set() method/Function is used for **type casting**.

```
# typecasting list to set  
s = set(["a", "b", "c"]) #here list is converted to set  
print(s)  
PS C:\Users\masrath parveen\onedrive\desktop> python f.py  
{'b', 'a', 'c'}
```

5. Name any two types of sorting algorithms.

SORTING

Sorting is the process of arranging data in a specific order, usually ascending or descending, based on a particular criterion.

TYPES OF SORTING ALGORITHMS

- Bubble Sort
- Selection Sort
- Merge Sort,
- Quick Sort etc...

6. Explain the use of the .get() method in dictionaries.

.get() Method Is Used To Accessing Dictionary Items(values)

We can access a value from a dictionary by using the **key** within square brackets or by using get() method. If the key is not found, it returns None

```
# create dictionary using { }  
d= {"name": 'john', "Rollno": 1234, "marks": 98.5}  
print(d)  
# Access using key  
print(d["name"])  
# Access using get(): pass key as an argument  
print(d.get("rollno")) #output: 1234  
print(d.get("marks")) #output: 98.5  
print(d.get("Subject")) #output: None
```

7. How does list slicing work?

Python List Slicing

Python **list slicing** is fundamental concept that let us easily access specific elements in a list. We can use both positive and negative indexing for slicing.

Python List Slicing Syntax

list_name[start : end : step]

- Using **[:] & [::]** without specifying any start, end, or step returns all elements of the list.
- To get all the items from a specific position to the **end** of the list, we can **specify the start index** and leave the end blank.
- And to get all the items before a specific index, we can **specify the end index** while leaving **start** blank.
- To extract elements between two specific positions, **specify both the start and end indices**

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
# Get all elements in the list
print(a[::])    #[1, 2, 3, 4, 5, 6, 7, 8, 9]
print(a[:])     #[1, 2, 3, 4, 5, 6, 7, 8, 9]
# Get elements starting from index 2 to the end of the list
b = a[2:]       #[3, 4, 5, 6, 7, 8, 9]
print(b)
# Get elements starting from index 0 to index 3 (excluding 3th index)
c = a[:3]       #[1, 2, 3]
print(c)
# Get elements from index 1 to index 4 (excluding index 4)
b = a[1:4]      #[2, 3, 4]
print(b)
```

8. What is meant by —tuple unpacking—?

Tuple Unpacking:

Tuple packing is the process of extracting/assigning values from a tuple into separate variables

```
t=(1,2,3,'python') #here 1,2,3,'python' are packed into the tuple t
a,b,c,d=t #tuple unpacking
# here each variable on LHS gets corresponding value from the tuple t
print(a) #1
print(b) #2
print(c) #3
print(d) #python
# or
a,b,c,d=1,2,3,'python'
print(a) #1
print(b) #2
print(c) #3
print(d) #python
```

9. How is a multidimensional array created in MATLAB?

In MATLAB, a multidimensional array can be created in several ways, primarily by:

- **Directly indexing and assigning values:** You can extend a 2D array into a multidimensional one by assigning values to indices beyond the initial 2D plane. MATLAB automatically expands the array dimensions as needed.

Example:

```
A = zeros(2,2); % Start with a 2x2 matrix  
A(:,:,2) = [5 6; 7 8]; % Create a second page (3rd dimension)  
A(:,:,3) = [9 10; 11 12]; % Create a third page
```

- Using the cat function: The cat (concatenate) function allows you to concatenate arrays along a specified dimension. This is particularly useful for building multidimensional arrays from smaller ones.

Example:

```
B1 = [1 2; 3 4];  
B2 = [5 6; 7 8];  
B3 = [9 10; 11 12];  
B = cat(3, B1, B2, B3); % Concatenate along the 3rd dimension
```

- Using zeros, ones, rand, etc., with multiple dimensions: Functions designed for creating arrays can directly specify multiple dimensions during initialization.

Example:

```
C = zeros(2, 3, 4); % Creates a 2x3x4 array of zeros  
D = rand(5, 2, 3, 2); % Creates a 5x2x3x2 array of random numbers
```

10. Describe the difference between `.remove()` and `.discard()` in sets.

`remove(element)` removes the specified element from a set. If the element is not found, it raises a `KeyError`.

`discard(element)` also removes the element but does not raise an error if the element is not present.

```
s = {1, 2, 3}
s.remove(2)      # Removes 2
s.discard(4)     # Does nothing, no error raised
# s.remove(4) would raise KeyError If the element is not
present.
```

LONG ANSWER TYPE QUESTIONS

1. Write a Python program that demonstrates adding, removing, and modifying elements in a list.

Adding Elements into List:

We can add elements to a list using the following methods:

append(): Adds an element at the end of the list.

extend(): Adds multiple elements to the end of the list.

insert(): Adds an element at a specific position.

```
# Initialize an empty list
a = []
# Adding 10 to end of list
a.append(10)
print("After append(10) the list is:", a)
# Inserting 5 at index 0
a.insert(0, 5)
print("After insert(0, 5):", a)
# Adding multiple elements [15, 20, 25] at the end
a.extend([15, 20, 25])
print("After extend([15, 20, 25]):", a)
```

Updating Elements into List

We can change the value of an element by accessing it using its index.

```
a = [10, 20, 30, 40, 50]
print("list before updating:")
print(a)
# Change the second element
a[1] = 25
print("list after updating:")
print(a)
```

Removing Elements from List

We can remove elements from a list using:

remove(): Removes the first occurrence of an element.

pop(): Removes the element at a specific index or the last element if no index is specified.

del statement: Deletes an element at a specified index.or it can delete an entire list

Clear(): Empties the list

```
a = [10, 20, 30, 40, 50]
print(a) # Original list
a.remove(30) # Remove 30
print(a) # List after removing 30
popped_val = a.pop(1) # Remove element at index 1
print(popped_val) # Popped value
print(a) # List after pop
del a[0] # Delete element at index 0
print(a) # List after del
a.clear() # Clear all elements
print(a) # List after clear
del a # Delete entire list
print(a) # Error: 'a' no longer exists
```


2. Implement a program to search an element in a list using linear search and explain the steps.

Iterative Linear Search Algorithm

Definition: In a linear search, each element in the array or list is checked sequentially until the desired element is found or the end is reached.

```
# write a program/function to implement linear search(using Iteration)
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

arr = [70, 40, 30, 11, 57, 41, 25, 41, 52]
target = 41
result = linear_search(arr, target)
if result != -1:
    print(f"Element found at index {result}")
else:
    print("Element not found")

PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Element found at index 3
```

Working of Linear search

Let the elements of array are -

0	1	2	3	4	5	6	7	8
70	40	30	11	57	41	25	14	52

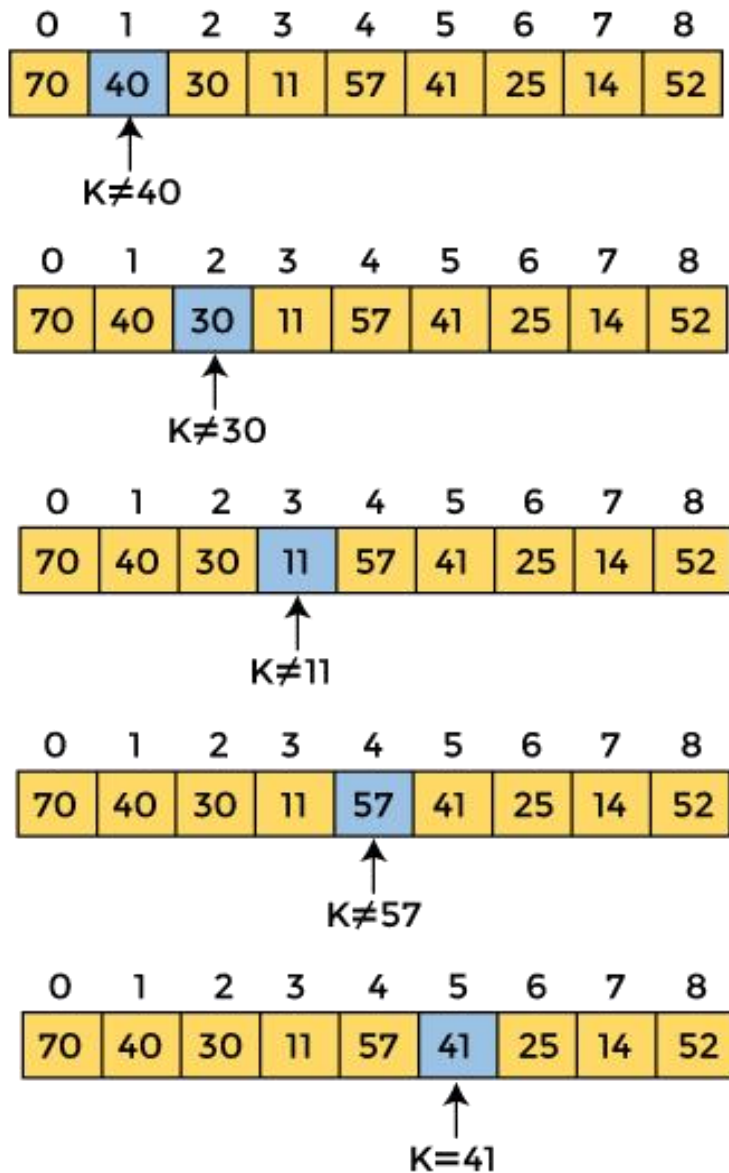
Let the element to be searched is $K = 41$

Now, start from the first element and compare K with each element of the array.

0	1	2	3	4	5	6	7	8
70	40	30	11	57	41	25	14	52

↑
 $K \neq 70$

The value of **K**, i.e., **41**, is not matched with the first element of the array.
So, move to the next element. And follow the same process until the
respective element is found.



Now, the element to be searched is found. So algorithm will return the index of the element match.

3. Compare and contrast the characteristics of sets and dictionaries in Python with examples.

Parameters	Set	Dictionary
Definition	A set is an unordered collection of unique elements.	A dictionary is an unordered collection of key-value pairs.
Syntax	Syntax includes curly brackets { , } with ',' separated data.	Syntax includes curly brackets { , } with ',' separated key-value data.
Creation	A set dictionary can be created using the set() function.	A dictionary can be created using the dict() function.
Empty Data Structure	An empty set can be created by s = set().	An empty dictionary can be created by {}.
Order	It is an unordered collection of data.	Ordered collection in Python version 3.7, unordered in Python Version=3.6.
Duplicate Data	All elements are unique in a Set.	Keys are unique, but two different keys CAN have the same value.
Indexing	Does NOT have an index based mechanism.	Has a Key based indexing i.e. keys identify the value.
Addition	The add() method adds an element to a set.	update() method updates specific key-value pair.
Deletion	Elements can be randomly deleted using pop().	pop(key) removes specified key along with its value.
Sorting	Unordered, so sorting is not advised.	Keys are sorted by using the sorted() method.
Search	Unordered, so searching is not applicable.	get(key) returns value against specified key.
Reversing	Unordered, so reverse is not advised.	No integer-based indexing, so no reversal.
Count	count() not defined for sets.	count() not defined for dictionaries.

```
# create dictionary using { } #DICTIONARY
d1 = {"name": 'john', "Rollno": 1234, "marks": 98.5}
print(d1)

# create dictionary using dict() constructor
d2 = dict(a = "apple", b = "banana", c = "mango")
print(d2)

# Access using key
print(d["name"])

# Access using get(): pass key as an argument
print(d.get("rollno"))
```

#SET:

```
s = { 17, 3.6,'python' ,True}
print(s)
s.add(40)
s.remove(17)
```

```
s1={1,2,3,4}
s2={3,4,5,6}

print(s1.union(s2))#unique elements {1, 2, 3, 4, 5, 6}
print(s1.intersection(s2))#common elements {3, 4}
print(s1|s2)#unique elements {1, 2, 3, 4, 5, 6}
print(s1&s2)#common elements {3, 4}
```

4. Assess the usage of MATLAB's zeros(), ones(), and eye() functions in matrix creation.

MATLAB provides several built-in functions to quickly create matrices with specific values. Three commonly used functions are zeros(), ones(), and eye(). These functions are essential in **initializing matrices**, especially in **numerical computations**, **simulations**, and **linear algebra operations**.

zeros() Function

Creates a matrix filled with **zeros**.

Useful for preallocating space for matrices during iterative computations.

Syntax:

```
Z = zeros(3, 2); % 3x2 matrix of all zeros
```

Output:

```
Z =  
    0    0  
    0    0  
    0    0
```

ones() Function

Creates a matrix filled with **ones**.

Commonly used in scaling, bias terms in ML, and matrix arithmetic (Adding constant values to matrices).

Syntax:

```
O = ones(2, 4); % 2x4 matrix of all ones
```

Output:

O =

```
1 1 1 1
1 1 1 1
```

eye() Function

Creates an **identity matrix** with 1s on the diagonal and 0s elsewhere.

Essential for solving linear systems, matrix inverses, and linear algebra.

Syntax:

```
I = eye(3); % 3x3 identity matrix
```

Output:

I =

```
1 0 0
0 1 0
0 0 1
```

The functions `zeros()`, `ones()`, and `eye()` provide **efficient and readable ways** to initialize matrices with predefined values. They are widely used in MATLAB for **scientific computing, algorithm design, and matrix operations**, improving both performance and code clarity.

5. Design a Python program that uses a dictionary to count the frequency of each word in a string.

```
# Input string
text = "This is a test. This test is only a test."
text = text.lower()          # Convert to lowercase
# Replace punctuation with space (just handling period
for simplicity)
text = text.replace(".", " ")
words = text.split()          # Split the string into
words
word_count = {}              # Create an empty
dictionary
for word in words:            # Count each word
    if word in word_count:
        word_count[word] += 1
    else:
        word_count[word] = 1
print("Word Frequency:")      # Print the result
for word in word_count:
    print(word, ":", word_count[word])
```

OUTPUT:

Word Frequency:

this : 2

is : 2

a : 2

test : 3

only : 1

6. Write a program to implement selection sort and display the sorted array.

```
# implement Selection Sort
def selection(arr):
    n = len(arr)
    for i in range(n):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]

arr = [64, 34, 25, 12, 22, 11, 90]
print("Original Array:", arr)

selection(arr)
print("Sorted Array:", arr)
OUTPUT:
Original Array: [64, 34, 25, 12, 22, 11, 90]
Sorted Array: [11, 12, 22, 25, 34, 64, 90]
```

7. Create a nested dictionary in Python and demonstrate how to access inner values.

Nested Dictionary

A dictionary inside another dictionary is called a nested dictionary.

It looks like this:

```
students = {
    "student1": {
        "name": "John",
        "age": 18,
        "marks": 90
    },
    "student2": {
        "name": "Mike",
        "age": 19,
        "marks": 95
    },
}
```



```
"student3": {  
    "name": "will",  
    "age": 17,  
    "marks": 65  
}  
}  
  
#access the entire dictionary dictionary  
print(students)  
  
#access values inside a nested dictionary  
print(students["student1"]["name"]) # Output: John  
print(students["student2"]["marks"]) # Output: 95
```